

Why I Built Inquirex: A Conversational DSL for Rules- Driven Questionnaires in Ruby

Table of Contents

The Problem	2
The Landscape: What Already Exists	2
State Machine Gems: AASM, Statesman, Workflow	2
Form Wizard Gems: Wicked, FormWizard, DVLA WizardFlow	3
Workflow Engines: FlowCore, workflow_rb	4
Rules Engines: Ruleby, Wongi Engine	4
Survey Gems: Surveyor, Questionnaire	4
The Gap	4
What Inquirex Does	5
The Conversational DSL	5
Data Types	7
The Rule System	8
JSON Serialization: The Bridge to Any Frontend	8
Auto-Answer and Pre-Population	9
LLM Integration: Intelligence as a DSL Verb	9
Architecture and Gem Ecosystem	10
A Real-World Example: Tax Intake	11
With LLM Enhancement	13
The Cross-Site Widget Model	13
Post-Processing: What Happens After the Last Answer	14
Getting Started	15
The Ecosystem Vision	16
Links	16
What's Next	16

Ruby has an embarrassment of riches when it comes to state machines and workflow

engines. What it **doesn't** have is a declarative, framework-agnostic engine for building **rules-driven, conditionally-branching questionnaires** — the kind of multi-step intake forms where the next question depends on the **answers** to previous ones. So I built one. Then I rebuilt it with a conversational DSL and an LLM integration layer. This is Inquirex.

Ask the right questions. Skip the rest.

This post explains why none of the existing gems solved the problem I actually had, what Inquirex does differently, and how a family of composable gems turns a Ruby DSL into a cross-platform questionnaire engine — from terminal to chat widget to Rails app — with optional LLM superpowers.

The Problem

My wife is a tax professional. Her practice needs to qualify inbound leads — quickly determine the complexity of a prospective client's tax situation and route them to the right service tier. This is a **branching questionnaire**, not a linear form:

- If you have business income ask how many businesses
- If more than two businesses ask for EIN details
- If you have crypto ask about exchanges and transaction volume
- If charitable contributions exceed \$5,000 ask for documentation

A simple W2 filer answers eight questions. A married filer with businesses, crypto, rentals, foreign accounts, and large charitable donations answers seventeen. **Same form, radically different paths through it.**

I looked at every gem I could find. None of them solved this.

The Landscape: What Already Exists

State Machine Gems: AASM, Statesman, Workflow

The most mature category. [AASM](#) is the 800-pound gorilla — actively maintained, 4,800+ stars, and battle-tested across thousands of Rails apps.

```
class Order
  include AASM
```

```

aasm do
  state :pending, initial: true
  state :paid, :shipped, :cancelled

  event :pay do
    transitions from: :pending, to: :paid, guard: :payment_valid?
  end

  event :ship do
    transitions from: :paid, to: :shipped
  end
end
end

```

AASM is **excellent** at what it does: modeling the lifecycle of a single object through well-defined states. Guards can conditionally allow or deny transitions.

[Statesman](#) from GoCardless adds database-backed transition history for auditability. The [workflow](#) gem offers a clean DSL with hooks and graphing support.

Why they don't solve the problem: These gems model **object lifecycle** — an order goes from `pending` to `paid` to `shipped`. They have no concept of **questions, answer collection**, or **data-dependent path selection across a graph of steps**. You could theoretically abuse a state machine into a questionnaire, but you'd be fighting the abstraction at every turn. The state lives on the **object**; in a questionnaire, the state is a **position in a directed graph** with accumulated context from previous answers driving the next transition.

Form Wizard Gems: Wicked, FormWizard, DVLA WizardFlow

[Wicked](#) is the go-to for multi-step forms in Rails. It's clean and well-integrated:

```

class AfterSignupController < ApplicationController
  include Wicked::Wizard
  steps :personal_info, :social_profiles, :confirmation

  def show
    @user = current_user
    render_wizard
  end

  def update
    @user = current_user
    @user.update(user_params)
    render_wizard @user
  end
end

```

[FormWizard](#) adds per-step validations and session persistence. The [DVLA WizardFlow](#) (from the UK Driver & Vehicle Licensing Agency) provides auto-generated routes from step definitions.

Why they don't solve the problem: All of these are **linear**. They define a fixed sequence of steps. Wicked lets you dynamically set steps at runtime, but there's no declarative rule DSL — no `contains()`, `all()`, `any()`, no composable

boolean conditions evaluated against accumulated answers. You **can** shove conditional logic into your controller, but then you're writing imperative spaghetti instead of a declarative flow definition.

Workflow Engines: FlowCore, workflow_rb

[FlowCore](#) is the most architecturally ambitious entry — a Petri-net-based workflow engine for Rails with ArcGuards, TransitionTriggers, and STI-based extensibility. It's genuinely impressive engineering.

Why it doesn't solve the problem: FlowCore is an **orchestration** engine. It routes **tasks** through a system — think approval chains, document review pipelines, business process automation. It has no concept of a user-facing questionnaire, collected answers, or conditional branching based on those answers. It's also tightly coupled to Rails and ActiveRecord by design.

Rules Engines: Ruleby, Wongi Engine

[Ruleby](#) implements the Rete algorithm for forward-chaining production rules. [Wongi Engine](#) provides a DSL for rule-based reasoning over fact triplets.

Why they don't solve the problem: These are pure inference engines — they evaluate rules over facts. They don't model a user-facing flow, don't collect answers step by step, and don't manage a stateful walk through a graph. You **could** use one as a rule evaluator inside a questionnaire engine, but on their own they're the wrong level of abstraction.

Survey Gems: Surveyor, Questionnaire

[Surveyor](#) is a Rails engine with its own survey DSL. The [Survey](#) gem stores questions in the database and manages attempts/scoring.

Why they don't solve the problem: Survey gems are designed for **data collection and scoring**, not for **conditional routing**. They store questions in a database and present them sequentially. Some support basic skip logic, but none offer a composable, AST-based rule system for arbitrarily complex branching conditions.

The Gap

Here's what I needed and couldn't find:

Capability	AASM	Wicked	FlowCore	Ruleby	Inquirex
Conversational DSL (ask/say/mention)					

Capability	AASM	Wicked	FlowCore	Ruleby	Inquirex
Directed graph (not linear)				N/A	
Conditional branching on collected data	¹		²		
AST-based composable rule system					
Typed data collection (11 types)					
JSON serialization (DSL → wire format)					
LLM integration as DSL verbs					
Framework-agnostic (no Rails dependency)					
Immutable flow definitions		N/A		N/A	
Pluggable UI adapters (TTY, JS, Rails)					

¹ AASM guards are method-based, not data-driven.

² FlowCore’s ArcGuards are extensible but don’t operate on accumulated user answers.

The gap is clear: no existing Ruby gem combines a conversational DSL, directed-graph traversal with conditional branching, an AST-based rule system, typed data collection, JSON serialization, and optional LLM integration — all without framework dependencies.

What Inquirex Does

Inquirex fills that gap. Here’s the thirty-second pitch:

Inquirex is a **conversational questionnaire engine** for Ruby. You define branching flows using verbs like `ask`, `say`, and `mention`. An **AST-based rule system** drives conditional transitions. A **stateful engine** collects typed answers. The definition **serializes to JSON** for consumption by any frontend — a terminal, a chat widget, a mobile app. And an optional **LLM layer** lets you add intelligence: **clarify**, **summarize**, **extract** — right inside the DSL.

The Conversational DSL

Inquirex reads like a conversation, because that’s what it **is**. The DSL uses verbs that map to the intent of each step:

Verb	Purpose	Collects Answer?
ask	Pose a question and collect a typed answer	Yes
say	Display an informational message	No
header	Section heading or title card	No
btw	Admonition, sidebar, or contextual bubble	No
warning	Alert the user to something important	No
confirm	Yes/no confirmation gate	Yes (boolean)

```

intake = Inquirex.define do
  start :welcome

  header :welcome do
    title "Tax Preparation Intake"
    subtitle "Let's understand your tax situation."
    transition to: :filing_status
  end

  ask :filing_status do
    type :enum
    question "What is your filing status for 2025?"
    options single: "Single",
             married_jointly: "Married Filing Jointly",
             married_separately: "Married Filing Separately",
             head_of_household: "Head of Household"
    transition to: :dependents
  end

  ask :dependents do
    type :integer
    question "How many dependents do you have?"
    default 0
    transition to: :income_types
  end

  btw :income_note do
    text "Select everything that applies – we'll ask for details only where needed."
    transition to: :income_types
  end

  ask :income_types do
    type :multi_enum
    question "Select all income types that apply to you."
    options W2: "W-2 Employment",
            "1099": "1099 Contract Work",
            Business: "Business Income",
            Investment: "Investment Income",
            Rental: "Rental Income",
            Retirement: "Retirement Income"
    transition to: :business_count,
               if_rule: contains(:income_types, "Business")
    transition to: :investment_details,
               if_rule: contains(:income_types, "Investment")
    transition to: :rental_details,
               if_rule: contains(:income_types, "Rental")
    transition to: :state_filing
  end
end

```

```

ask :business_count do
  type :integer
  question "How many businesses do you own or are a partner in?"
  transition to: :complex_business_info,
    if_rule: greater_than(:business_count, 2)
  transition to: :business_details
end

warning :complex_business_info do
  text "With more than 2 businesses, we'll need additional documentation."
  transition to: :business_ein
end

ask :business_ein do
  type :text
  question "Please provide your primary EIN and briefly describe each entity."
  transition to: :business_details
end

# ... remaining steps
end

```

Notice how header, btw, and warning create breathing room in the flow. The questionnaire doesn't feel like a machine-gun of inputs — it feels like a guided conversation.

Data Types

Every ask step declares a type. Inquirex ships with eleven core types that cover the vast majority of real-world intake forms:

Type	Description	Example Value
:string	Short text (single line)	"Alice Smith"
:text	Long text (multiline)	"EIN: 12-345. Alpha LLC..."
:integer	Whole number	3
:decimal	Floating point number	45000.50
:currency	Monetary amount (2 decimal places)	12500.00
:boolean	True/false	true
:enum	Single selection from a list	"married_jointly"
:multi_enum	Multiple selections from a list	["W2", "Business", "Rental"]
:date	Calendar date	"2025-04-15"
:email	Email address (validated format)	"alice@example.com"
:phone	Phone number	"+1-555-123-4567"

Types serve two purposes: **validation** (the engine can reject a non-numeric value for an :integer field) and **rendering hints** (a frontend adapter knows to show a date picker for :date, a dropdown for :enum, checkboxes for :multi_enum).

The Rule System

Rules are **AST objects** — not strings, not hashes, not procs. They're immutable, composable, serialize to JSON, and evaluate polymorphically:

```
# Atomic rules
contains(:income_types, "Business") # Array inclusion
equals(:status, "married_jointly")  # Equality
greater_than(:count, 2)              # Numeric comparison
less_than(:age, 18)                  # Numeric comparison
not_empty(:dependents)               # Presence check

# Composite rules - nest arbitrarily
all(
  equals(:filing_status, "married_jointly"),
  any(
    greater_than(:business_count, 3),
    contains(:income_types, "Rental")
  ),
  not_empty(:dependents)
)
```

Every rule implements `evaluate(answers)` where `answers` is a hash of `{ step_id ⇒ value }`. No magic, no `method_missing` gymnastics, no runtime metaprogramming. Just polymorphic dispatch on immutable data structures.

Why AST objects instead of procs or lambdas?

1. **Serializable** — you can dump a rule tree to JSON and reconstruct it on any platform
2. **Inspectable** — `rule.to_s` gives you a human-readable representation
3. **Composable** — `all()` and `any()` nest without limit
4. **Cross-platform** — the JavaScript widget evaluates the same rule JSON client-side
5. **Exportable** — the Mermaid exporter can label graph edges with rule descriptions

JSON Serialization: The Bridge to Any Frontend

This is where Inquirex diverges fundamentally from every other Ruby questionnaire gem. The entire flow definition — steps, rules, transitions, types, options — **serializes to JSON**:

```
definition = Inquirex.define { ... }

# Serialize to JSON (rules become AST descriptions, lambdas are stripped)
json = definition.to_json

# Reconstruct from JSON (round-trip fidelity for all non-lambda elements)
restored = Inquirex::Definition.from_json(json)
```

The JSON wire format is the contract between the Ruby backend and any frontend. A transition in JSON looks like:


```
{
  "to": "business_count",
  "rule": { "op": "contains", "field": "income_types", "value": "Business" },
  "requires_server": false
}
```

A JavaScript widget can evaluate `contains`, `equals`, `greater_than` — the basic rule operators — entirely client-side. Steps marked `"requires_server": true` (like LLM-powered steps) round-trip to the server. This means the questionnaire is **instant** for pure branching and only incurs network latency when intelligence is needed.

Auto-Answer and Pre-Population

Real-world questionnaires don't always start from zero. Inquirex provides three distinct mechanisms:

```
ask :state do
  type :enum
  question "Which state do you live in?"
  default { geo_lookup(request.ip) } # Pre-fill, but user can change
end

ask :spouse_income do
  type :currency
  question "What is your spouse's annual income?"
  skip_if equals(:filing_status, "single") # Skip entirely
end

ask :total_businesses do
  type :integer
  compute { |answers| answers.dig(:business_details).values.sum } # Auto-calculate
end
```

`default` pre-populates but allows override. `skip_if` removes the step from the path entirely. `compute` auto-calculates from existing answers and never shows the step to the user. Three concepts, three verbs, no ambiguity.

LLM Integration: Intelligence as a DSL Verb

The `inquirex-llm` extension adds verbs that invoke a language model as part of the questionnaire flow:

```
# Provided by the inquirex-llm gem
ask :business_description do
  type :text
  question "Describe your business in a few sentences."
  transition to: :business_extracted
end

clarify :business_extracted do
  from :business_description
  prompt "Extract structured business information from the user's description."
```

```

schema industry: :string,
        entity_type: :enum,
        employee_count: :integer,
        estimated_revenue: :currency
  transition to: :investment_details
end

summarize :intake_summary do
  from_all true
  prompt "Summarize this client's tax situation and flag any complexity concerns."
  transition to: :review
end

```

LLM Verb	Purpose
clarify	Extract structured data from a free-text answer
describe	Generate a natural-language description from structured data
summarize	Produce a summary of all (or selected) answers collected so far
detour	Dynamically generate follow-up questions based on an answer

LLM steps are **always server-side** — API keys never touch the frontend. The JSON serialization marks these steps with `requires_server: true`, and the frontend shows a "thinking" indicator while the round-trip completes.

Every LLM verb requires a `schema` declaration — the expected output structure. This isn't optional. LLM responses are validated against the schema before insertion into the answers. If validation fails, the engine retries or falls back to a default.

Architecture and Gem Ecosystem

Inquirex is a family of composable gems with clear dependency boundaries:

```

inquirex      (core: DSL, rules, engine, types, serialization)
├── inquirex-ui (rendering metadata: widget hints, responsive layouts)
│   ├── inquirex-tty (terminal adapter via TTY Toolkit)
│   ├── inquirex-js (embeddable JavaScript chat widget)
│   └── inquirex-rails (Rails Engine: persistence, API, asset serving)
└── inquirex-llm (LLM verbs: clarify, describe, summarize, detour)

```

Gem	Responsibility
<code>inquirex</code>	Pure Ruby core — DSL, rules, engine, types, JSON serialization. Zero dependencies.
<code>inquirex-ui</code>	Adds widget DSL verb for rendering hints (radio groups, dropdowns, etc.). Framework-agnostic metadata — produces enriched JSON, not HTML.
<code>inquirex-tty</code>	Terminal questionnaire adapter using <code>TTY::Prompt</code> . Maps data types to TTY widgets. Developer tool for testing and demos.

Gem	Responsibility
<code>inquirex-js</code>	Self-contained JavaScript widget. Embeds on any site via a <code><script></code> tag. Renders as a floating chat bubble (Intercom-style). Evaluates pure rules client-side; round-trips to server for LLM steps.
<code>inquirex-rails</code>	Rails Engine. ActiveRecord persistence for definitions and answers. API endpoints for serving definitions and receiving submissions. Asset pipeline integration for the JS widget.
<code>inquirex-llm</code>	LLM integration layer. Adds <code>clarify</code> , <code>describe</code> , <code>summarize</code> , <code>detour</code> verbs to the DSL. Requires a server-side LLM provider (OpenAI, Anthropic, etc.). Validates LLM output against declared schemas.



The core has **zero UI logic**, **zero DB logic**, and **zero framework dependencies**. It's pure Ruby, pure data structures, pure functions. The UI gem adds rendering metadata. The adapter gems (TTY, JS, Rails) handle actual rendering, persistence, and input collection. The LLM gem extends the DSL vocabulary without touching the core engine.

A Real-World Example: Tax Intake

Here's the flow I actually built for my wife's practice — a 17-step tax intake wizard:

```
tax_intake = Inquirex.define do
  start :filing_status
```

```

header :welcome do
  title "2025 Tax Preparation Intake"
  subtitle "Answer a few questions so we can prepare your return efficiently."
  transition to: :filing_status
end

ask :filing_status do
  type :enum
  question "What is your filing status for 2025?"
  options single: "Single",
          married_jointly: "Married Filing Jointly",
          married_separately: "Married Filing Separately",
          head_of_household: "Head of Household"
  transition to: :dependents
end

ask :dependents do
  type :integer
  question "How many dependents do you have?"
  default 0
  transition to: :income_types
end

ask :income_types do
  type :multi_enum
  question "Select all income types that apply to you in 2025."
  options W2: "W-2 Employment",
          "1099": "1099 Contract Work",
          Business: "Business Income",
          Investment: "Investment Income",
          Rental: "Rental Income",
          Retirement: "Retirement Income"
  transition to: :business_count,
    if_rule: contains(:income_types, "Business")
  transition to: :investment_details,
    if_rule: contains(:income_types, "Investment")
  transition to: :rental_details,
    if_rule: contains(:income_types, "Rental")
  transition to: :state_filing
end

ask :business_count do
  type :integer
  question "How many total businesses do you own or are a partner in?"
  transition to: :complex_business_info,
    if_rule: greater_than(:business_count, 2)
  transition to: :business_details
end

warning :complex_business_info do
  text "With more than 2 businesses, we'll need additional information."
  transition to: :business_ein
end

ask :business_ein do
  type :text
  question "Please provide your primary EIN and a brief description of each entity."
  transition to: :business_details
end

# ... remaining steps follow the same pattern
end

```

The path lengths vary dramatically:

Scenario	Steps	Skipped
Single filer, W2 only	8	9 steps skipped
Married, business + investments	12	5 steps skipped
Head of household, rentals + foreign accounts	10	7 steps skipped
Maximum complexity (all income types, high charitable)	17	Nothing skipped

Same flow definition. Four completely different user experiences.

With LLM Enhancement

Using `inquirex-llm`, the same flow gains intelligence:

```
# After collecting all answers...
summarize :intake_summary do
  from_all true
  prompt "Based on this client's tax situation, provide:
    1) An estimated complexity tier (Simple, Moderate, Complex)
    2) Estimated preparation fee range
    3) Any red flags or items requiring special attention"
  schema complexity_tier: :enum,
    fee_range_low: :currency,
    fee_range_high: :currency,
    red_flags: :text
  transition to: :review
end
```

The LLM step runs server-side, validates its output against the declared schema, and inserts structured data into the answers hash. The user sees the result as a summary card in their chat widget. The tax professional sees it as actionable intake data.

The Cross-Site Widget Model

The centerpiece of the Inquirex ecosystem is the **embeddable chat widget**. A client site includes one script tag:

```
<script src="https://qualified.at/widget.js" data-site-id="abc123"></script>
```

The widget:

1. Fetches the flow definition JSON from the qualified.at API
2. Renders as a floating chat bubble in the bottom-right corner
3. Walks the user through the questionnaire with the appropriate widget per type
4. Evaluates pure branching rules **client-side** (no latency)
5. Round-trips to the server only for LLM-powered steps
6. Submits completed answers back to the API
7. Triggers server-side post-processing (lambdas, callbacks, storage)

The flow definition is authored in Ruby, serialized to JSON, served via API, and consumed by JavaScript. The Ruby DSL is the source of truth. The JSON is the wire format. The widget is one of many possible frontends.

Post-Processing: What Happens After the Last Answer

When a questionnaire is complete, the answers come back to the server as a JSON hash. The Ruby DSL — which contains elements that were stripped from the JSON (lambdas, compute steps, callbacks) — rehydrates and runs post-processing:

```
Inquirex.define do
  # ... flow steps ...

  compute :total_entities do
    type :integer
    calculate { |answers| answers.dig(:business_details).values.sum }
  end

  on_complete do |answers|
    # Store in database
    TaxIntake.create!(
      client_name: answers[:contact_name],
      client_email: answers[:contact_email],
      complexity_tier: answers[:complexity_tier],
      data: answers.to_json
    )

    # Send notification
    IntakeMailer.new_submission(answers).deliver_later

    # Push to spreadsheet
    GoogleSheets.append(ENV["INTAKE_SHEET_ID"], answers.to_flat_hash)
  end
end
```

The callback/store pattern means persistence is pluggable. The core gem doesn't know or care whether answers end up in PostgreSQL, an email, a Google Sheet, or a webhook. That's the adapter's job.

Getting Started

Install the gem:

```
gem install inquirex
```

Or add to your Gemfile:

```
gem "inquirex"  
gem "inquirex-tty" # optional: terminal-based questionnaires  
gem "inquirex-llm" # optional: LLM integration
```

Define a flow, run it, export it:

```
require "inquirex"  
  
definition = Inquirex.define do  
  start :name  
  
  ask :name do  
    type :string  
    question "What is your name?"  
    transition to: :role  
  end  
  
  ask :role do  
    type :enum  
    question "What's your role?"  
    options developer: "Developer",  
            designer: "Designer",  
            manager: "Manager"  
    transition to: :languages,  
              if_rule: equals(:role, "developer")  
    transition to: :tools,  
              if_rule: equals(:role, "designer")  
    transition to: :team_size  
  end  
  
  ask :languages do  
    type :multi_enum  
    question "Which languages do you use?"  
    options ruby: "Ruby", python: "Python",  
            javascript: "JavaScript", go: "Go", rust: "Rust"  
    transition to: :done  
  end  
  
  ask :tools do  
    type :multi_enum  
    question "Which tools do you use?"  
    options figma: "Figma", sketch: "Sketch",  
            photoshop: "Photoshop", illustrator: "Illustrator"  
    transition to: :done  
  end  
  
  ask :team_size do  
    type :integer
```

```

    question "How large is your team?"
    transition to: :done
  end

  say :done do
    text "Thanks! We'll be in touch."
  end
end

# Run in terminal (requires inquirex-tty)
Inquirex::TTY::Runner.new(definition).run

# Export to JSON (for the JS widget or any frontend)
puts definition.to_json

# Export as Mermaid diagram
puts Inquirex::Graph::MermaidExporter.new(definition).export

```

The Ecosystem Vision

Inquirex is the **engine**. What I'm building on top of it is the **car**:

Qualified.at — a service that lets non-technical users define branching intake forms through a visual editor, embed them on any website via a chat widget, and receive structured qualification data with optional LLM-powered analysis. Think Typeform, but with **real** conditional branching logic driven by an AST rule system, not just "skip to section 3."

The gem ecosystem is MIT-licensed and free forever. Qualified.at is the revenue layer.

Links

- [Inquirex on GitHub](#) (**migrating from flowengine-rb**)
- [Inquirex on RubyGems](#)
- [Qualified.at](#) — the hosted platform

What's Next

I'm actively working on:

- **inquirex-js** — the embeddable chat widget for any website

- **inquirex-rails** — Rails Engine with ActiveRecord persistence and API endpoints
- **inquirex-llm** — LLM integration with schema-validated structured output
- **Visual flow editor** — a React-based DAG editor that exports Inquirex DSL
- **Qualified.at** — the hosted platform built on top of all of the above

If you're building intake forms, qualification wizards, onboarding flows, or any kind of conditionally-branching questionnaire in Ruby — give Inquirex a look. It might save you from reinventing the wheel.

And if you have ideas, bug reports, or want to contribute — [the issue tracker is open](#).

Konstantin Gredeskoul is a software engineer, 4x CTO, and the author of 30+ open-source Ruby gems with over 120M cumulative downloads. He lives in San Francisco and builds things at [reinvent.one](#).